Clase 3 - Anexo: interpretación de errores en R y Python

Joselina Davyt-Colo

Anexo: interpretación de errores

Complemento del Punto 1 de la Clase 3
Institución: Facultad de Ciencias Empresariales y Economía — Universidad de Montevideo

Qué es este anexo

El Punto 1 de la clase presentó tres mensajes de error. Este anexo los extiende a un catálogo de consulta.

No está pensado para dictarse completo: se usa **durante el práctico**, cuando aparece un mensaje que no se entiende.

Cubre R y Python en paralelo, porque los dos lenguajes fallan de maneras distintas y esa diferencia es en sí misma un contenido de la materia.

Parte I. Cómo se lee un mensaje

Los tres desenlaces posibles

Cuando una instrucción no hace lo que se esperaba, hay tres finales distintos:

Desenlace	Qué hace el programa	Cómo se detecta
Error	Se detiene y no devuelve nada	Solo: la pantalla lo muestra
Advertencia	Sigue, devuelve algo y avisa	Leyendo la salida completa
Resultado sin sentido	Sigue, devuelve algo y no avisa	Únicamente si uno lo controla

Importante

El primero interrumpe el trabajo. El tercero es el único que llega al informe final.

Cada lenguaje calla en lugares distintos

Situación	Python	R
Convertir `"45,0"` a número	Se detiene (`ValueError`)	Devuelve `NA` con advertencia
Pedir el elemento 5 de un vector de 3	Se detiene (`IndexError`)	Devuelve `NA`, sin aviso
Pedir una columna que no existe	Se detiene (`KeyError`)	Devuelve `NULL`, sin aviso
Sumar una columna que no existe	Se detiene	Devuelve `0`, sin aviso

Situación	Python	R
Archivo con una fila de más	Se detiene (`ParserError`)	Lo lee mal y sigue
Promediar con datos faltantes	**Los ignora y devuelve un número**	**Devuelve `NA`**

Advertencia

La regla no es “R es permisivo y Python es estricto”. La última fila va en sentido contrario: ante datos faltantes, R se detiene en `NA` y Python devuelve un promedio calculado sobre las filas que quedaron. Cada lenguaje tiene su propia zona de silencio.

Anatomía del mensaje en Python

```
Traceback (most recent call last):
  File "analisis.py", line 12, in <module>
    promedio = df["peso"].mean()
  File "analisis.py", line 4, in cargar
    return pd.read_csv(ruta)
ValueError: could not convert string to float: '45,0'
```

Parte	Qué aporta
`ValueError`	La categoría del problema
`could not convert...`	Qué pasó, en una frase
`File ..., line 12`	Dónde pasó
`promedio = df[...]`	La línea exacta

Se lee de abajo hacia arriba. La última línea dice *qué*; las de arriba, *por dónde llegó*.

Anatomía del mensaje en R

R no arma un traceback: escribe una sola línea con dos piezas separadas por `:`.

```
Error in edad + 1 : non-numeric argument to binary operator
```

Parte	Qué aporta
`Error in edad + 1`	**Dónde**: la operación que falló
`non-numeric argument...`	**Qué**: la descripción del problema

Cuando el problema no está en ninguna operación en particular, la primera pieza desaparece:

```
Error: object 'datos' not found
```

Las advertencias de R se leen aparte

Las advertencias no interrumpen nada y **pueden aparecer al final**, lejos de la línea que las produjo:

```r
warnings()          # lista las advertencias acumuladas
traceback()         # muestra el camino del último error
```

Nota

Un script de R puede terminar "bien" y haber dejado veinte advertencias por el camino. Correr `warnings()` después de cada bloque largo es parte del oficio.

Las tres preguntas

Frente a cualquier mensaje, en cualquiera de los dos lenguajes:

1. **¿De qué tipo es el problema?** ¿Un nombre, un tipo, un valor, una posición, un archivo?
2. **¿Sobre qué objeto ocurrió?** El mensaje casi siempre lo nombra.
3. **¿Qué esperaba encontrar el programa y qué encontró en su lugar?**

Las tres se responden leyendo. Ninguna se responde reescribiendo la línea al azar.

Parte II. Catálogo por causa

Seis causas cubren casi todo lo que aparece en el primer mes.

A. El nombre no existe

Python	R
`NameError: name 'datos' is not defined`	`Error: object 'datos' not found`
`ModuleNotFoundError: No module named 'pandas'`	`Error in library(tidyverse) : there is no package called 'tidyverse'`
`AttributeError: 'DataFrame' object has no attribute 'colums'`	`Error in meanx(x) : could not find function "meanx"`

Qué revisar, en este orden:

1. ¿La celda que creaba el objeto se ejecutó en esta sesión?
2. ¿Está escrito exactamente igual? `Peso`, `peso` y `PESO` son tres nombres distintos en los dos lenguajes.
3. ¿El paquete está instalado, o está instalado pero sin cargar? Son dos problemas distintos con soluciones distintas.

El caso de `df` en R

```
df$peso
# Error in df$peso : object of type 'closure' is not subsettable
```

`closure` significa **función**. El mensaje dice: "le pediste una columna a algo que es una función".

En R, `df` ya existe: es la densidad de la distribución F. Si el objeto propio nunca se creó, el nombre no queda vacío, queda apuntando a la función de R.

Advertencia

Nombres a evitar en R porque ya están ocupados: `df`, `data`, `c`, `mean`, `T`, `F`.
En R conviene llamar a los datos `datos` o `encuesta`, no `df`.

B. El código no se puede leer

Acá el programa ni siquiera llegó a ejecutar: no entendió la escritura.

```python
# Python
print("hola"
# SyntaxError: '(' was never closed
```

```r
# R, en un script
mean(x
# Error: unexpected end of input
```

```r
# R, un símbolo donde no corresponde
x <- c(1 2)
# Error: unexpected numeric constant
```

La consola colgada

Si falta cerrar un paréntesis o una comilla, la consola **no da error**: queda esperando el resto.

Lenguaje	Señal
R	El prompt cambia de `>` a `+`
Python	El prompt cambia de `>>>` a `...`

No es un error: es una pregunta. Se sale con `Esc` (R) o `Ctrl + C` (Python) y se vuelve a escribir la línea completa.

La indentación en Python

```python
if peso > 50:
print(peso)
# IndentationError: expected an indented block after 'if' statement on line 1
```

Python usa el **espacio en blanco** para delimitar bloques; R usa llaves `{ }`. Un error de sangría en Python es un error real; en R, apenas un problema estético.

Nota

Configurar el editor para que inserte cuatro espacios en lugar de tabulaciones elimina toda una familia de errores.

C. El tipo no corresponde

```python
edad = "35"
edad + 1
# TypeError: can only concatenate str (not "int") to str
```

```r
edad <- "35"
edad + 1
# Error in edad + 1 : non-numeric argument to binary operator
```

Los dos mensajes dicen lo mismo: **hay un texto donde el programa esperaba un número**.

El origen casi nunca está en esa línea. Está antes: en una lectura de archivo que dejó la columna como texto.

El método no existe para ese tipo

```python
peso = 45.0
peso.upper()
# AttributeError: 'float' object has no attribute 'upper'
```

Dos lecturas posibles, y conviene descartarlas en este orden:

1. El nombre del método está mal escrito (`.uper()` en lugar de `.upper()`).
2. El objeto no es lo que se creía. Un `AttributeError` sobre un objeto que debería ser un DataFrame suele significar que se recibió otra cosa.

D. El valor no se puede convertir

Este es el punto donde los dos lenguajes se separan de verdad.

```python
float("45,0")
# ValueError: could not convert string to float: '45,0'
```

```r
as.numeric("45,0")
# [1] NA
# Warning message: NAs introduced by coercion
```

El tipo era el correcto (los dos recibieron texto, que es lo que esperaban). El **contenido** no se puede interpretar como número, porque la coma decimal no es el separador que el lenguaje espera.

Por qué esta diferencia importa

Python **se detiene** y obliga a resolverlo. R **devuelve `NA`** y sigue.

Un archivo con quinientas filas y decimales con coma, leído sin corregir:

- En Python: el proceso se interrumpe en la primera conversión.
- En R: se obtienen quinientos `NA`, un promedio que es `NA` y una advertencia de una línea que es fácil pasar por alto.

Importante

`NAs introduced by coercion` es la advertencia más importante de R para quien trabaja con datos: significa que se perdieron valores en silencio.

E. La posición o la columna no existe

Operación	Python	R
Elemento fuera de rango	`IndexError: list index out of range`	Devuelve `NA`
Columna inexistente	`KeyError: 'pesoo'`	`d$pesoo` devuelve `NULL`
Selección explícita	—	`d[, "pesoo"]` sí da error: `undefined columns selected`

```r
x <- c(10, 20, 30)
x[5]
# [1] NA        <- no hay error
```

Recordar además que **el primer elemento es `x[1]` en R y `x[0]` en Python**: pedir `x[0]` en R devuelve un vector vacío, no el primer valor.

El `$` de R hace coincidencias parciales

```r
datos <- data.frame(peso = c(45, 120.5))
datos$pes
# [1] 45.0 120.5     <- devolvió la columna 'peso'
```

R completa el nombre si no hay ambigüedad. Es cómodo y es peligroso: un error de tipeo puede devolver la columna equivocada sin ningún aviso.

Advertencia

Usar `datos[["peso"]]` en lugar de `datos$peso` evita la coincidencia parcial. En Python el problema no existe: `df["pes"]` da `KeyError`.

F. El archivo: cinco fallas distintas

Qué pasó	Python	R
No está donde se lo buscó	`FileNotFoundError: [Errno 2] No such file or directory`	`cannot open file 'datos.csv': No such file or directory`
Separador equivocado	Una sola columna con todo adentro	Una sola columna, y a veces filas renombradas
Decimal equivocado	Columna de texto (`object`)	Columna `chr`
Codificación equivocada	`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf1`	Advertencia y texto deformado: `Se\361or`
Fila con un campo de más	`ParserError: Expected 3 fields in line 3, saw 4`	Lee mal y sigue

Las cinco se resuelven con argumentos de lectura, no reescribiendo el archivo.

El archivo no está donde se lo buscó

```r
from pathlib import Path
Path.cwd()                          # dónde estoy parado
list(Path.cwd().iterdir())          # qué hay acá
```

```r
getwd()                          # dónde estoy parado
list.files()                     # qué hay acá
```

Si el archivo aparece en el listado, el problema es el nombre. Si no aparece, el problema es la carpeta.

Separador, decimal y codificación

```python
pd.read_csv("datos.csv", sep=";", decimal=",", encoding="latin-1")
```

```r
read.csv("datos.csv", sep = ";", dec = ",", fileEncoding = "latin1")
```

Tres argumentos, tres problemas independientes:

- `sep` decide **dónde termina cada columna**.
- `decimal` / `dec` decide **si la coma separa decimales o campos**.
- `encoding` / `fileEncoding` decide **cómo se traducen las eñes y las tildes**.

El caso más insidioso: la fila de más

```
a,b,c
1,2,3
4,5,6,7      <- una coma de más
```

```python
pd.read_csv("malo.csv")
# ParserError: Error tokenizing data. C error: Expected 3 fields in line 3, saw 4
```

```r
read.csv("malo.csv")
#   a b  c
# 1 2 3 NA
# 4 5 6  7
```

R no protesta: usa la primera columna como nombres de fila y devuelve una tabla que parece correcta. Los datos están corridos un lugar.

Importante

Este es el argumento central del anexo: el mensaje de error de Python es una molestia de treinta segundos; el silencio de R puede sobrevivir hasta la presentación final.

Parte III. Los errores que no dan error

Siete fallas silenciosas

Situación	Qué se obtiene	Cómo se detecta
Columna numérica leída como texto	`.sum()` concatena: `"45,0120,5"`	Revisar los tipos antes de calcular
Columna inexistente en R	`sum(d$pesoo)` devuelve `0`	Comparar contra `names(d)`
Coincidencia parcial en R	Devuelve otra columna	Usar `[["nombre"]]`

Situación	Qué se obtiene	Cómo se detecta
Faltantes al promediar	Python los ignora; R devuelve `NA`	Contar los faltantes primero
Reciclaje de vectores en R	Repite valores para completar	Comparar `length()` de los dos vectores
Resultado no asignado	Nada cambia	Verificar `shape` / `dim` después
Celdas ejecutadas fuera de orden	Un objeto viejo con nombre nuevo	Reiniciar y correr todo de arriba abajo

La suma que concatena

```python
pd.read_csv("pesos.csv", sep=";")["peso"].sum()
# '45,0120,5'
```

No hay error. Hay una cadena de texto que pegó los dos valores uno detrás del otro.

```python
pd.read_csv("pesos.csv", sep=";")["peso"].mean()
# TypeError: Cannot perform reduction 'mean' with string dtype
```

El promedio sí protesta. La suma no. **La misma columna mal leída produce un error o un disparate según qué se le pida.**

La columna que no existe suma cero

```r
datos <- data.frame(peso = c(45, 120.5))
sum(datos$pesoo)
# [1] 0
```

`datos$pesoo` devuelve `NULL`, y la suma de nada es cero. Un cero es un resultado perfectamente creíble en un informe.

Verificación de un segundo:

```r
names(datos)
```

El promedio con faltantes

```python
df["peso"].mean()     # 82.75  -> ignoró el faltante
```

```r
mean(datos$peso)      # NA     -> se detuvo en el faltante
mean(datos$peso, na.rm = TRUE)   # 82.75
```

Los dos resultados son defendibles, pero **son decisiones distintas sobre los mismos datos**. Python la toma por defecto; R obliga a escribirla.

Importante

Ninguna de las dos es “la correcta”. Lo incorrecto es no saber cuál se tomó. Esto se registra en la bitácora.

El resultado que no se asignó

```
df.dropna()           # devuelve una tabla nueva; df queda igual
df = df.dropna()      # ahora sí
```

```
subset(datos, !is.na(peso))          # se imprime y se pierde
datos <- subset(datos, !is.na(peso)) # ahora sí
```

Síntoma típico: “corrí la línea y no cambió nada”. No falló: el resultado se calculó, se mostró y se descartó.

El reciclaje de R

```
c(1, 2, 3, 4) + c(1, 2)
# [1] 2 4 4 6      <- repitió el vector corto, sin aviso

c(1, 2, 3) + c(1, 2)
# [1] 2 4 4
# Warning: longer object length is not a multiple of shorter object length
```

R avisa solo cuando los largos no son múltiplos. Cuando sí lo son —el caso más frecuente en la práctica— la operación se completa en silencio.

Comparar contra un faltante

```
x <- c(1, NA, 3)
x == NA
# [1] NA NA NA      <- ni TRUE ni FALSE

is.na(x)
# [1] FALSE TRUE FALSE
```

Un faltante no es igual ni distinto de nada: es desconocido. Filtrar con `== NA` devuelve una máscara de `NA` que no selecciona ninguna fila.

Parte IV. Procedimiento de diagnóstico

Cinco pasos, en orden

1. **Leer el mensaje entero** y separar las dos piezas: qué y dónde.
2. **Verificar el objeto**: si existe, qué tipo tiene, qué contiene.
3. **Reducir**: encontrar la línea mínima que reproduce el problema.
4. **Comparar** lo que hay con lo que se esperaba: tipos, cantidad de filas, faltantes.
5. **Registrar** el mensaje y la causa en la bitácora antes de seguir.

El paso 5 no es burocracia: el mismo error vuelve a aparecer tres semanas después.

Comandos de inspección

Pregunta	Python	R
¿Existe el objeto?	`"df" in globals()`	`exists("datos")`
¿Qué es?	`type(x)`	`class(x)`
¿Qué tiene adentro?	`df.info()`	`str(datos)`
¿De qué tipo es cada columna?	`df.dtypes`	`sapply(datos, class)`
¿Cuántas filas y columnas?	`df.shape`	`dim(datos)`
¿Cómo empieza?	`df.head()`	`head(datos)`
¿Cuántos faltantes hay?	`df.isna().sum()`	`colSums(is.na(datos))`
¿Qué columnas tiene?	`df.columns`	`names(datos)`

`df.info()` y `str(datos)` son los dos comandos más rentables del curso: contestan cinco preguntas a la vez.

Cómo buscar un mensaje

El mensaje contiene nombres propios que solo existen en la máquina de quien lo escribió. Buscar así no sirve:

```
KeyError: 'peso_kg_2024_final'
```

Buscar así sí:

```
pandas KeyError columna que existe en el csv
```

Regla: conservar el tipo de error y la parte genérica del mensaje; borrar los nombres de archivos, columnas y rutas.

El ejemplo mínimo reproducible

Antes de pedir ayuda —a un compañero, a un buscador o a un modelo de lenguaje— conviene reducir el problema a algo que otro pueda ejecutar:

```python
import pandas as pd
d = pd.DataFrame({"peso": ["45,0", "120,5"]})
d["peso"].mean()     # TypeError
```

Tres líneas, sin archivos externos, sin datos reales. **Muchas veces el problema aparece solo al armar este ejemplo**: la reducción es en sí misma el diagnóstico.

Parte V. Ejercicios

Ejercicio 1: opción múltiple

Una sola respuesta correcta por pregunta. Justificar en la bitácora las que la consigna indique.

Preguntas 1 a 4

1. En R, `Error in df$peso : object of type 'closure' is not subsettable` significa que:

a. El data frame `df` está vacío

b. `df` nunca fue creado y el nombre corresponde a una función de R

c. La columna `peso` no existe en `df`

d. Falta cargar el paquete que define `$`

2. En Python, `float("45,0")` se detiene con `ValueError`. En R, `as.numeric("45,0")` devuelve `NA` con una advertencia. Esto significa que:

a. R interpretó bien la coma decimal y Python no

b. Los dos fallaron en lo mismo; solo Python interrumpió el trabajo

c. R convirtió el valor y Python no pudo

d. Son dos operaciones distintas y no se pueden comparar

3. Un script de R calcula `sum(datos$pesos)` y devuelve `0`. La columna del archivo se llama `peso`. Lo que ocurrió es que:

a. Todos los pesos valen cero

b. `datos$pesos` devolvió `NULL` y la suma de nada es cero

c. R convirtió los pesos a texto

d. El archivo se leyó vacío

4. Sobre los mismos datos con un faltante, `df["peso"].mean()` devuelve `82.75` y `mean(datos$peso)` devuelve `NA`. Esto se explica porque:

a. R leyó mal el archivo

b. Python descarta los faltantes por defecto y R no

c. Los dos lenguajes calculan promedios distintos

d. Falta convertir la columna en Python

Preguntas 5 a 8

5. `ParserError: Expected 3 fields in line 3, saw 4` indica que:

a. El archivo tiene cuatro columnas y se pidieron tres

b. La fila 3 tiene un separador de más respecto del encabezado

c. El separador declarado es el equivocado

d. La fila 3 está vacía

6. `UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf1` aparece porque:

a. El archivo está dañado

b. El archivo está guardado en otra codificación y contiene eñes o tildes

c. Falta instalar un paquete de idioma

d. El separador decimal es una coma

7. En R, `x <- c(10, 20, 30)` y luego `x[5]`. El resultado es:

a. Un error de índice fuera de rango

b. `NA`, sin ningún aviso

c. El último elemento, `30`

d. Un vector vacío

8. En R, `datos$pes` devuelve la columna `peso`. Esto ocurre porque:

a. R corrigió el error de tipeo

b. `$` completa nombres parciales cuando no hay ambigüedad

c. Las dos columnas existen y son iguales

d. `peso` y `pes` son el mismo nombre para R

Preguntas 9 a 12

9. Un alumno ejecuta `df.dropna()`, no aparece ningún error y `df` sigue teniendo las mismas filas. La causa es que:

a. No había faltantes en los datos

b. El resultado se calculó y no se asignó a ningún nombre

c. `dropna()` no funciona en esa versión de pandas

d. Faltaba ejecutar la celda anterior

10. `IndentationError: expected an indented block after 'if' statement on line 1` no tiene equivalente en R porque:

a. R no tiene estructuras condicionales

b. R delimita los bloques con llaves y no con espacios

c. R corrige la sangría automáticamente

d. En R la sangría también es obligatoria, pero avisa distinto

11. La consola de R muestra `+` en lugar de `>` y no responde. Lo que corresponde hacer es:

a. Reiniciar la sesión y perder los objetos creados

b. Cancelar con `Esc`: quedó una comilla o un paréntesis sin cerrar

c. Escribir `+` de nuevo para confirmar

d. Volver a instalar el paquete que se estaba usando

12. En R, `c(1, 2, 3, 4) + c(1, 2)` devuelve `2 4 4 6` sin ninguna advertencia porque:

a. R sumó solo los dos primeros elementos

b. R repitió el vector corto hasta igualar el largo, y los largos son múltiplos

c. R convirtió los dos vectores a un solo número

d. Es un error que se muestra recién al imprimir

Clave de corrección

	Correcta	El error que detecta
1	**b**	Suponer que un nombre no usado está vacío

	Correcta	El error que detecta
2	**b**	Leer la advertencia de R como si fuera un éxito
3	**b**	Aceptar un cero creíble sin verificar el nombre
4	**b**	No ver que el tratamiento de faltantes es una decisión
5	b	Confundir un archivo mal formado con una mala lectura
6	b	Atribuir al archivo un problema de codificación
7	**b**	Esperar que R avise cuando se sale de rango
8	**b**	Confiar en el nombre escrito sin verificar cuál se devolvió
9	b	Suponer que toda operación modifica el objeto
10	b	No distinguir sintaxis de convención
11	b	Leer una pregunta del intérprete como un error
12	**b**	Esperar un error donde hay reciclaje silencioso

Las seis en negrita corresponden a fallas silenciosas: conviene discutirlas en clase.

Ejercicio 2: diagnóstico

Para cada síntoma, indicar **causa probable** y **el comando de una línea que la confirma**.

a. Un promedio de salarios da `NA` en R. El archivo tiene 1.200 filas y ningún salario vacío a simple vista.

b. Una tabla leída en Python tiene una sola columna, cuyo nombre es `id;fecha;monto`.

c. `df["monto"].sum()` devuelve un texto larguísimo en lugar de un número.

d. Un data frame de R se lee sin error, pero tiene una columna menos que el archivo original y los nombres de fila se ven raros.

e. El mismo script funciona en la máquina de un compañero y falla en la propia con `FileNotFoundError`.

Clave del ejercicio 2

	Causa probable	Confirmación
a	Decimales con coma: la columna es texto o quedó en `NA`	`str(datos)` o `sum(is.na(datos$salario))`
b	Separador equivocado: el archivo usa `;`	`pd.read_csv(ruta, nrows=3)` y mirar `df.columns`

	Causa probable	Confirmación
c	La columna es texto y la suma concatena	`df.dtypes`
d	Alguna fila tiene un campo de más y R corrió los datos	`dim(datos)` y `head(datos)` con los nombres de fila
e	Ruta absoluta con el usuario del compañero	`getwd()` / `Path.cwd()` y `list.files()`

Ejercicio 3: para la bitácora

Provocar deliberadamente **seis errores**, tres en cada lenguaje, tomados de causas distintas del catálogo.

Para cada uno, registrar:

1. La línea que lo produjo
2. El mensaje completo, copiado tal cual
3. Las respuestas a las tres preguntas de la Parte I
4. La corrección

Nota

Un error provocado a propósito y entendido cuesta dos minutos. El mismo error encontrado durante el práctico cuesta media hora.

Resumen operativo

Si el mensaje dice…	Mirar…
`not defined` / `not found`	El orden de ejecución de las celdas
`Syntax` / `unexpected` / prompt `+`	Paréntesis y comillas sin cerrar
`non-numeric` / `TypeError`	Los tipos de las columnas
`could not convert` / `NAs introduced`	El separador decimal
`KeyError` / `NULL` / `undefined columns`	Los nombres reales de las columnas
`FileNotFoundError` / `cannot open`	El directorio de trabajo
`Expected N fields` / filas raras	El archivo crudo, en un editor de texto
Nada	Los tipos, el largo y la cantidad de faltantes

La última fila es la que exige el curso.